ISAMC 2026 · Conference Presentation

A Multi-Protocol Cyber-Deception Framework Combining Language-Model Response Synthesis with Blockchain-Anchored Forensic Logging

A honeypot that talks back like a real server — and whose evidence cannot be silently edited.

Radhika G, Sisir Reddy Nellore, Pranav Sharma Chouduru, Surya Tejeswar Reddy Nagireddy,
Guru Sai Prasad Reddy Bompalli, Vishnu Sathwick Rajala

Department of Computer Science and Engineering · Amrita School of Computing, Coimbatore · Amrita Vishwa Vidyapeetham, India

What this talk covers

1	**The problem**	Honeypots are easy to unmask, and their logs are easy to tamper with
2	**Our idea**	Let a language model write replies; anchor every record's hash on a blockchain
3	**The system**	Five decoys · three-tier response path · intent classifier · on-chain logging · dashboard
4	**Evaluation**	2,500 real attack payloads · latency, throughput, diversity, realism · tamper test
5	**Honest findings**	Where it works, where it is weak (AI-API decoy), and 3 engineering bugs we fixed
6	**Comparison & future**	Versus Cowrie, ShelLM, HoneyLLM · limitations · next steps

First, what is a honeypot?

The plain-English version

Think of a decoy house

- A fake server put on the network on purpose
- It has no real users — so anyone who touches it is, by definition, suspicious
- Every command the attacker types is recorded
- Defenders learn: who is attacking, what tools they use, what they want
- Cheap, low risk, high-signal — a standard sensor in security teams

How today's cheap honeypots work

- They do NOT run a real operating system (too risky, too costly)
- They pretend, using a hand-written table of canned answers:
  - attacker types ls → reply: "file1 file2"
  - attacker types whoami → reply: "root"
- Examples: Cowrie, Kippo (SSH), Dionaea (malware capture)
- Anything NOT in the table → wrong, missing, or identical reply

Problem 1 — Static honeypots are easy to unmask

A finite answer table can always be enumerated

- A scanner sends a few unusual commands and watches the replies
- Replies come from a finite, deterministic set → identical across 'different' hosts
- Conclusion in seconds: "this is a decoy" → the attacker leaves
- Bigger tables do not help — any static set is still enumerable
- **This is a structural flaw, not a tuning problem**

What is needed: a GENERATIVE reply mechanism — one that can produce a plausible, in-character answer to a request it has never seen before.

1 packet

Vetterl & Clayton (USENIX WOOT 2018) fingerprinted low/medium-interaction honeypots with a single packet

2,844

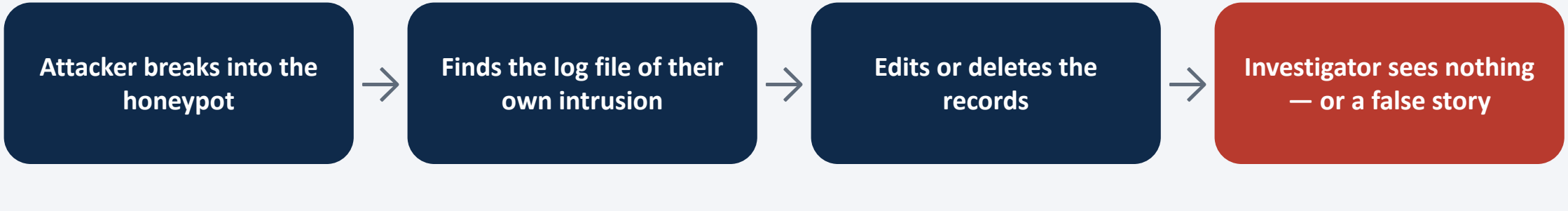
real Cowrie / Kippo honeypots identified on the public Internet

EER 0.0183

Equal-error rate — the detector was wrong less than 2% of the time

Problem 2 — The evidence lives on the machine being attacked

A log file is only as trustworthy as the host that holds it



- A honeypot's whole purpose is to be attacked — so we must assume it CAN be compromised
- If the only copy of the audit trail is on that host, the attacker can rewrite history
- For logs to count as forensic evidence they need an integrity guarantee that survives compromise of the logger itself
- **Requirement: tamper-EVIDENCE that does not depend on trusting the honeypot host**

Our idea — fix both problems in one framework

Realism → a language model writes the replies

- Replace the fixed table with an LM that emulates the service
- Answers any request, keeps session context, no finite set to enumerate
- Tiered: fast table → cache → LM only for novel input

Integrity → hash every record onto a blockchain

- Each interaction → SHA-256 digest → transaction on an Ethereum chain
- Later: re-hash the stored record, compare with the on-chain value
- Any edit changes the hash → tampering is detected

Three contributions of the paper

C1	**Multi-protocol LM-backed deception layer**	5 decoys (SSH, SQL, AI API, Ethereum RPC, CI/CD) behind one proxy, with a three-tier response path
C2	**Blockchain-anchored forensic logging**	Every record hashed and committed on-chain — tamper-evidence independent of the honeypot host
C3	**Four-axis honest evaluation**	2,500 real payloads · latency / concurrency / diversity / realism · failure modes reported explicitly

Threat model and design goals

Who are we fooling, and what must the system guarantee?

Two kinds of adversary

- **Automated scanner**
  - High-rate bot, stereotyped requests, moves on fast
  - Detects decoys via deterministic / inconsistent replies
  - Stresses: breadth, consistency, latency uniformity
- **Interactive intruder**
  - Human at a shell, exploring at human speed
  - Issues novel, context-dependent commands
  - Stresses: in-character answers to unseen input
- **Plus: the honeypot host itself may be compromised**

Six design goals (each is measured later)

G1	Non-enumerable responses	→ diversity
G2	Handle never-seen input in character	→ realism
G3	Latency like a real service	→ latency
G4	Cover multiple protocols	→ 5 decoys
G5	Host-independent log integrity	→ tamper test
G6	Explainable attack-intent labels	→ classifier

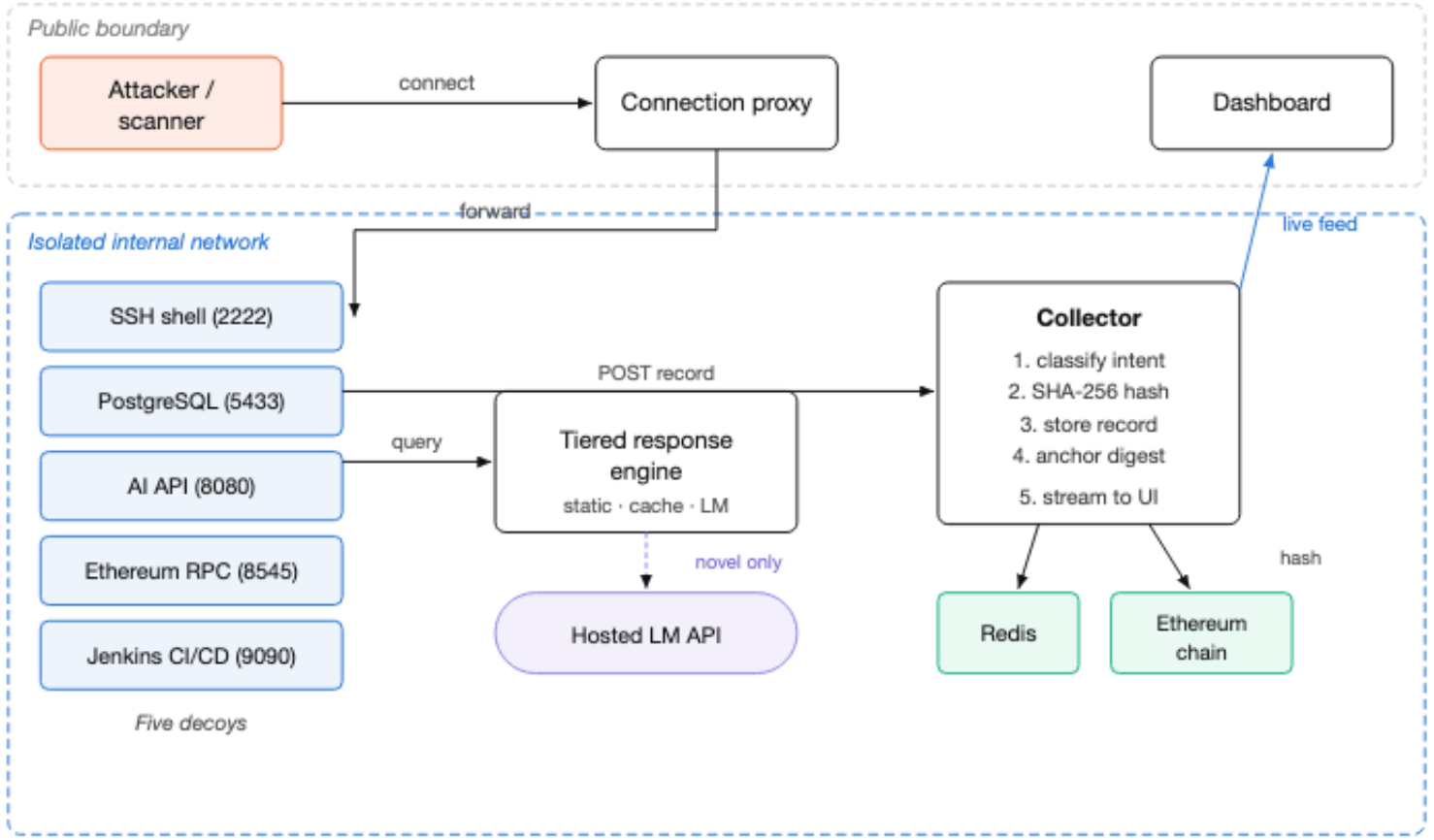
Part 2

System architecture

Five decoys · one proxy · a tiered response engine · a collector that classifies, hashes and anchors

Architecture at a glance

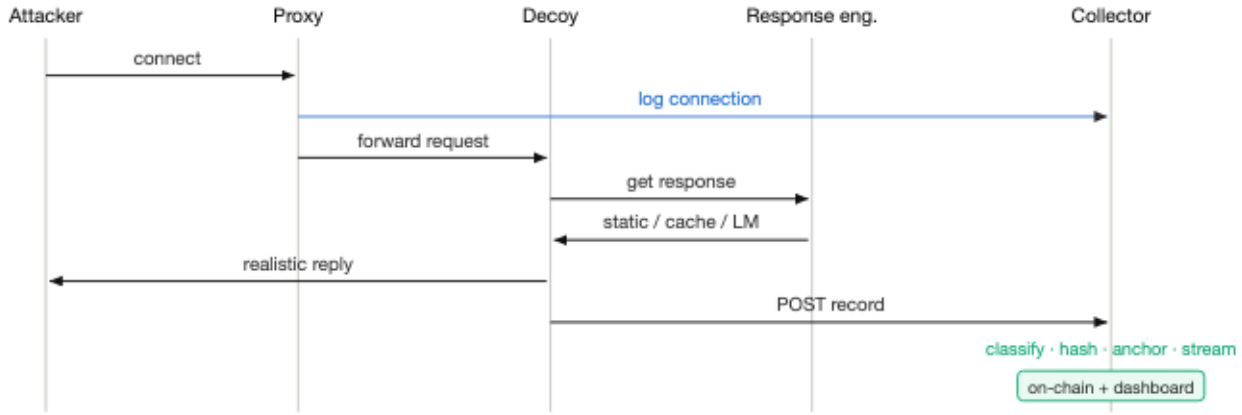
Only the proxy and the dashboard face the outside; everything else lives on an isolated internal network



- **Public boundary**
  - Connection proxy — the single entry point
  - Dashboard — operator's view
- **Isolated internal network**
  - 5 decoy services
  - Tiered response engine (static · cache · LM)
  - Collector: classify → hash → store → anchor → stream
  - Redis datastore + local Ethereum chain
- **Attacker can never reach the control plane directly**

The connection proxy and one interaction's journey

Log first, respond second — even a bare port scan is captured



- One asynchronous proxy listens on all five public ports and forwards each connection to the right internal decoy
- Records source address + target port BEFORE any decoy responds → a scan that never completes a handshake is still logged
- Single choke point for connection-level telemetry; decoys stay off the public interface
- Then: decoy → tiered engine → reply; collector → classify → SHA-256 → store → anchor on-chain → stream to dashboard

Five decoys — five modern attack surfaces

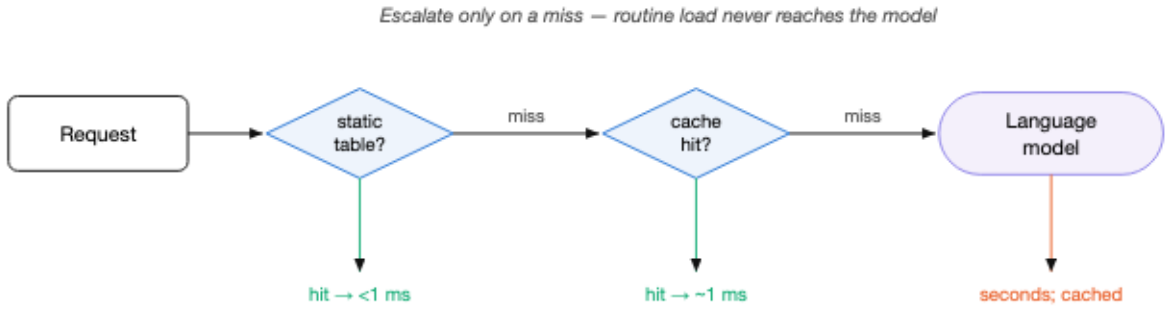
Prior LLM honeypots cover one surface (a Linux shell); we cover five

Decoy	Port	What it pretends to be	What it baits
SSH shell	2222	Corporate Ubuntu database host; accepts any password; interactive shell	Brute-force logins, recon, reverse shells, priv-esc
PostgreSQL	5433	SQL endpoint returning plausible tables or realistic errors	SQL injection
Internal AI API	8080	'Internal' chat / prediction service	Prompt injection, jailbreaks, XXE / SSRF probes
Ethereum JSON-RPC	8545	Exposed blockchain node	Wallet enumeration & draining (crypto theft)
Jenkins CI/CD	9090	Build server with script endpoints	Supply-chain / CI-CD abuse, RCE

Each decoy forwards a normalised record of every interaction to the central collector.

The three-tier response path

Why not send every request to the AI? Because seconds of delay is itself a tell — and it costs money



Tier 1 · Static fast-path — < 1 ms

Common enumeration commands (ls, whoami, ifconfig…) from a fixed table. Low realism cost — these really do have stable outputs on a real host.

Tier 2 · Response cache — ~1 ms

Seen this request (plus recent context) before? Serve from in-memory cache — a repeated probe never pays for a second LM call.

Tier 3 · LM synthesis — seconds

Only novel requests. Llama-3.1-8B via hosted API, service-specific persona: 'emit ONLY the raw output stream — no explanation, no markdown'. Session history for consistency.

Intent classification — labelling what the attacker wants

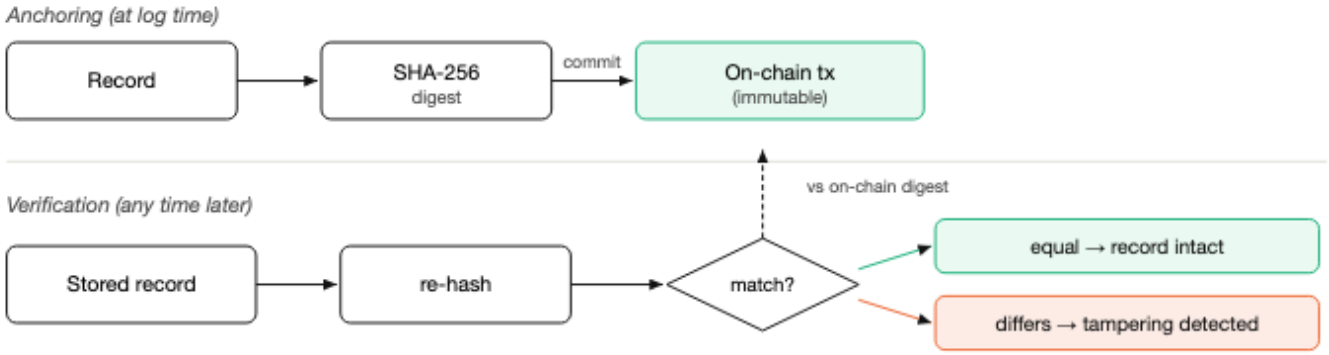
Rule-based on purpose: auditable, zero latency, no training data

SQL injection	**Prompt injection**	**Crypto theft**	**Cryptomining**
Reverse shell / RCE	**Privilege escalation**	**Supply-chain / CI-CD**	**Reconnaissance**

- Every interaction gets ONE of eight categories (else 'unknown'), from transparent rules on the raw request text
- Why rules and not machine learning? Rules are auditable, explainable, need no labelled dataset, and add zero latency — the right trade for a forensic sensor
- The language model does response SYNTHESIS only — classification is a separate, deterministic stage
- Fun fact for later: our first rule set left ~75% of traffic 'unknown'. We fixed it (Gap 3).

Blockchain-anchored logging — making the evidence tamper-evident

Only a 32-byte fingerprint goes on-chain; the full record stays in the datastore

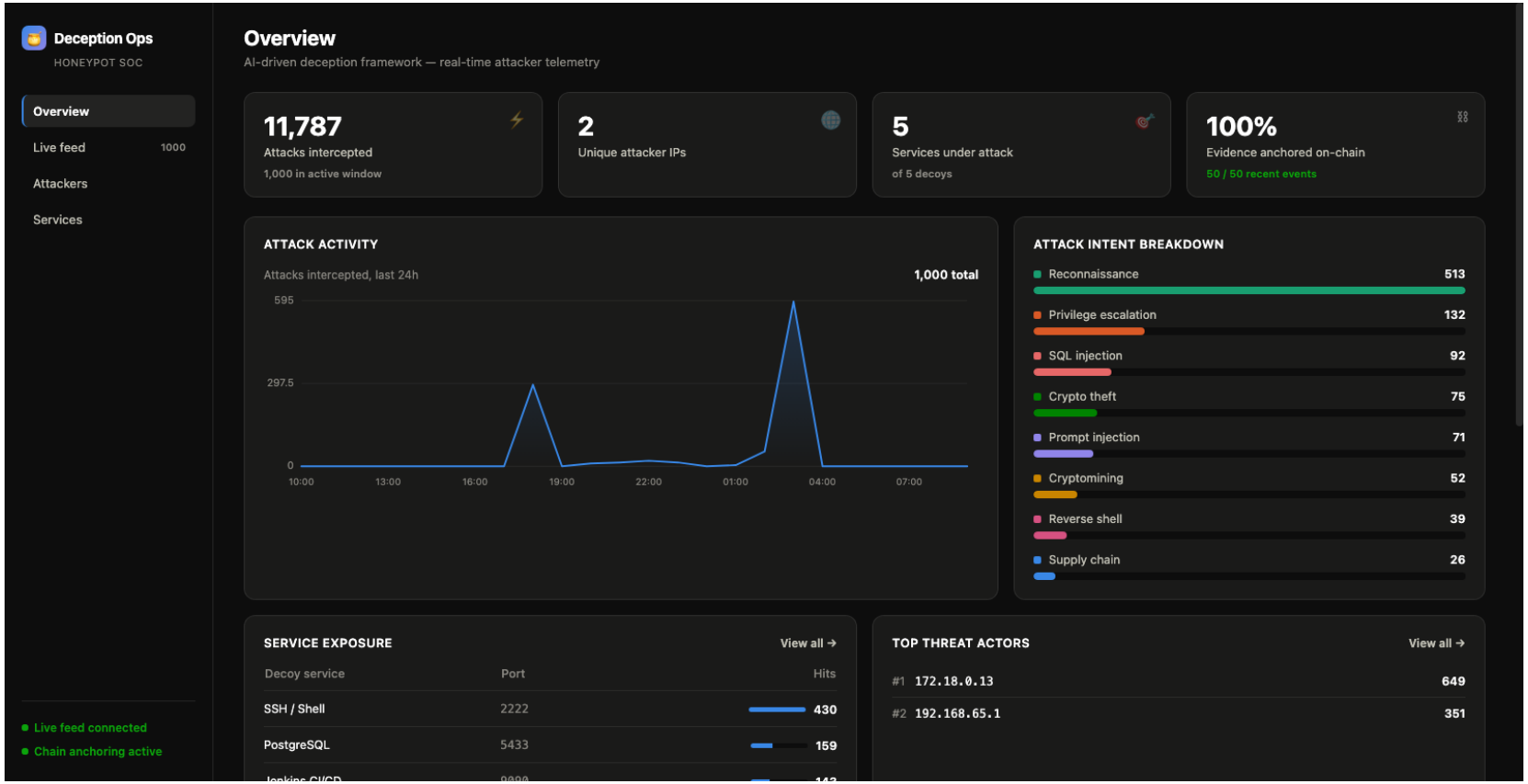


- **At log time**
  - Serialise the record deterministically
  - Compute the SHA-256 digest (32 bytes)
  - Send a transaction carrying the digest to a local Ethereum chain
  - Full record stays in the datastore
- **At verification — any time, by anyone**
  - Fetch the stored record, re-hash it
  - Compare with the on-chain digest
  - **Equal → intact. Different → tampered.**

Why it holds: the on-chain digest cannot be silently rewritten. Any edit to the stored record changes its SHA-256, so the mismatch is detectable by anyone — integrity no longer depends on trusting the honeypot host.

Real-time operator dashboard

Live capture during operation — intent breakdown, per-service exposure, top attackers, anchoring status



- Attacks intercepted, unique attacker IPs, services under attack
- % of evidence anchored on-chain
- Attack-intent breakdown across the 8 categories
- Per-attacker drill-down: services touched, severity-weighted threat rating, intent mix, timeline of real payloads
- Searchable live feed: payload, decoy reply, SHA-256 digest, on-chain status per event

Part 3

Evaluation

Four independent axes on 2,500 real attack payloads — reported separately, because they trade against each other

Evaluation methodology

Real payloads only — nothing model-generated, nothing hand-invented

2,500

unique real attack payloads from SecLists & PayloadsAllTheThings, each with a provenance field

5 partitions

1,862 SSH · 474 SQL · 136 API · 18 Web3 · 10 CI/CD (capped at real payloads available)

8B → 70B

Llama-3.1-8B generates replies; an independent Llama-3.1-70B judges realism

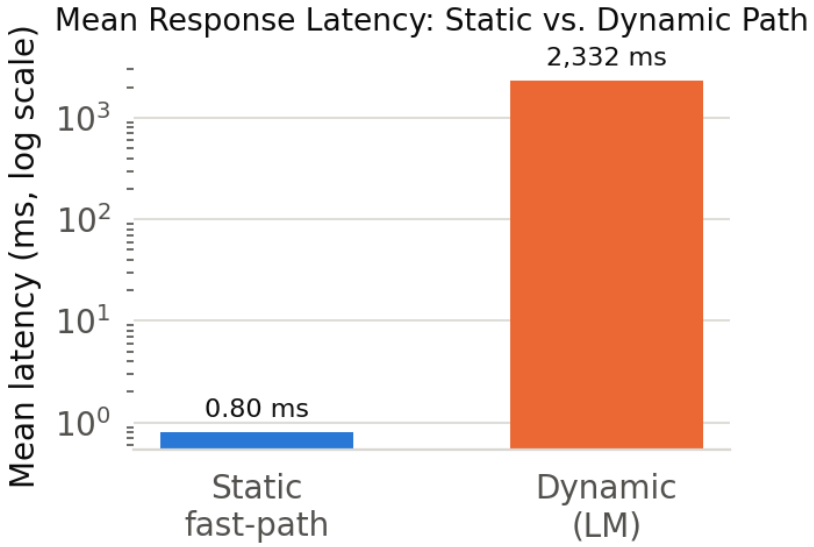
vs static

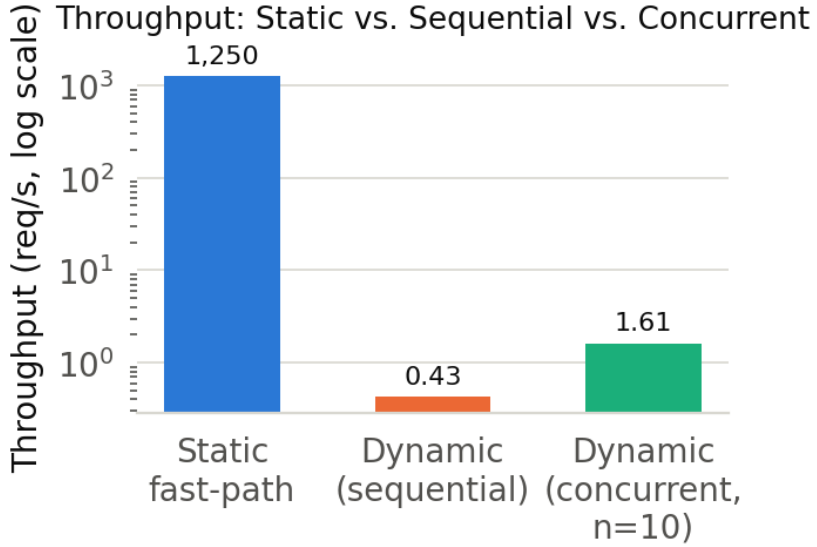
baseline: a fixed response table (Cowrie-style)

Axis 1 · Latency & sequential throughput	Per-request latency of static path vs LM path over repeated trials — mean, median, std-dev. Throughput = 1 / mean latency.
Axis 2 · Concurrent throughput	10 simultaneous sessions. Each request carries a fresh nonce → guaranteed cache miss → real LM call (see Gap 2!).
Axis 3 · Response diversity	Uniqueness ratio = distinct outputs ÷ total outputs, per service. Direct proxy for fingerprintability. Static table → ~0; generative → ~1.
Axis 4 · LM-judge realism	Llama-3.1-70B rates each (payload, reply) pair 1–5: 1 = obvious tell (template / refusal / AI disclaimer), 5 = indistinguishable. n = 30 sampled.
+ Blockchain integrity test	Fetch record → re-hash → matches on-chain. Modify one field → re-hash → must NOT match.

Result 1 — Latency and throughput

Four orders of magnitude between the tiers — this gap IS the justification for the tiered design





0.80 ms

static fast-path mean latency (~1,250 req/s)

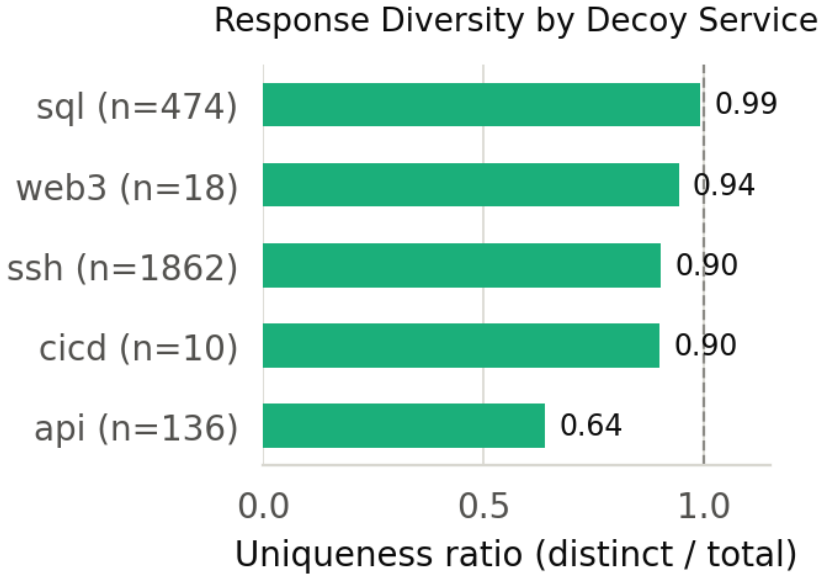
2.33 s

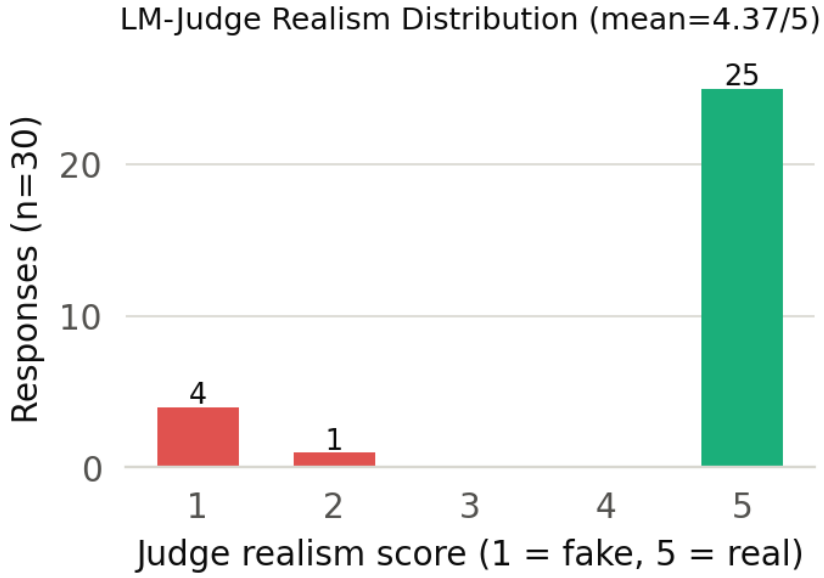
dynamic LM path mean (median 1.73 s, σ 1.40 s) → 0.43 req/s

- Under 10 concurrent sessions (cache misses forced): 1.61 req/s sustained, 2.19 s mean per-request latency
- Dynamic latency is dominated by the round-trip to the hosted inference API, not local computation
- Routine traffic (~80%) never pays the LM cost — it is answered at static-honeypot speed

Result 2 — Diversity and realism

Can an attacker cluster our replies into a small set? And do they look real?





0.64 – 0.992

uniqueness ratio across all five decoys (static table → ~0)

4.37 / 5

mean LM-judge realism (σ 1.45, n = 30); 25 of 30 rated a full 5

- Diversity: SQL 0.992 · Web3 0.944 · SSH 0.903 · CI/CD 0.900 · AI-API 0.64 — all far from the near-zero of a static table
- Realism distribution is bimodal: mostly 'clearly realistic', a few 'clearly unconvincing', almost none in between — usually in character, occasionally lapses
- The low ratings concentrate on the AI-API decoy (generic error text for XML/SSRF probes) plus a few over-terse SSH 'command not found' replies

The honest per-service picture

Two independent metrics agree on which decoy is weakest — that agreement is itself evidence the finding is real

Decoy	Payloads (n)	Distinct outputs	Uniqueness ratio	Judged (n)	Realism (mean/5)
SQL (PostgreSQL)	474	470	0.992	3	5.00
Web3 (Ethereum RPC)	18	17	0.944	—	—
SSH (shell)	1862	1681	0.903	24	4.71
CI/CD (Jenkins)	10	9	0.900	—	—
AI API	136	87	0.640	3	1.00
Overall	**2500**	**—**	**0.64–0.99**	**30**	**4.37**

Web3 and CI/CD have small payload counts because their real attack surface is a finite set of RPC methods / exploit endpoints (Sec. V-A), and were below the judge-sampling threshold; their high uniqueness still holds. Per-service realism samples are small and are reported to show the direction and magnitude of the disparity, not as precise point estimates.

Structured-protocol decoys: strong

- SQL, Web3, SSH, CI/CD: diversity 0.90–0.99, realism 4.71–5.00
- Rigid output grammars (SQL errors, JSON-RPC schema) leave the LM little room to break character

AI-API decoy: the one soft spot

- Diversity 0.64, judged realism 1.00 (n = 3)
- Likely cause: an 'internal AI API' is under-constrained → model falls back to generic error text
- Concrete fix: a tightly specified output contract for that decoy

Result 3 — Blockchain integrity test

Goal G5: does tampering get detected? Yes — directly verified

Untampered record

- Fetch stored record → recompute SHA-256
- Digest matches the anchored on-chain value exactly ✓

After altering ONE field

- Re-hash the modified record
- Digest diverges from the on-chain value → tampering detected ✓

What this proves: an attacker who compromises the honeypot and edits a stored record cannot make the edited record's hash match what was already committed on-chain. The audit trail's integrity is verifiable independently of the logging host — by any party, at any later time.

Three engineering gaps we found — and fixed — during construction

Reported deliberately: two of them are exactly the kind of error that silently corrupts an evaluation

Gap 1 · LM latency made the naïve design non-viable

The gap: Every SSH command went to the model → ~4.0 s mean.

Diagnosis: Measured, not guessed: fresh TCP+TLS connection per request; oversized generation cap.

Fix: Tiered path + one pooled client + tuned cap → ~1.35 s (≈3× faster). Pooling across all 5 decoys cut latency 30–41% and halved variance. A smaller model did NOT help — provider routing dominated.

Gap 2 · The benchmark was measuring the cache, not the model

The gap: Concurrent throughput looked like hundreds of req/s.

Diagnosis: Reused payloads from the diversity pass → every 'concurrent' request was a Redis cache hit.

Fix: Append a fresh nonce per request → forced cache miss. Honest figure: 1.61 req/s — 2–3 orders of magnitude lower. Kept in the paper on purpose.

Gap 3 · The intent classifier left ~75% of traffic 'unknown'

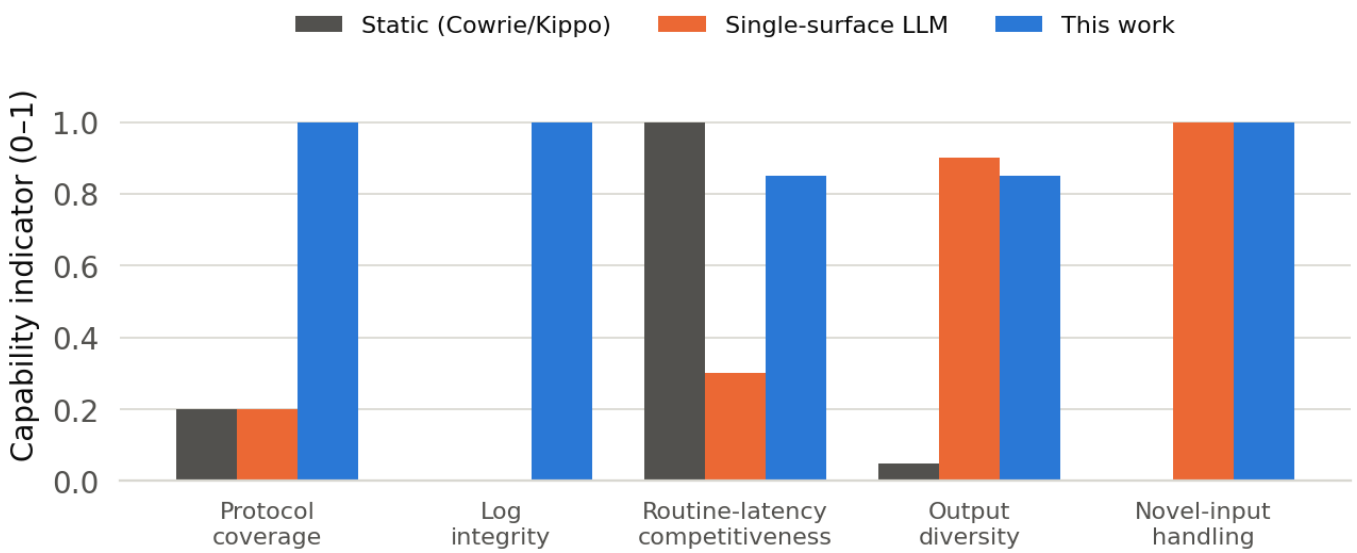
The gap: Sensor output dominated by 'unknown' = no actionable intelligence.

Diagnosis: Rules too narrow: missed proxy probes, first-contact shell enumeration, wallet-draining RPC, brute-force logins, compact injections.

Fix: Added crypto-theft category, broadened recon & priv-esc, widened injection/RCE patterns → unknown 75% → ~0% on 1,000 held-out real payloads.

How we compare with prior systems

No single prior system is multi-protocol, generative, fast on routine traffic AND integrity-anchored



- Static (Cowrie/Kippo): ~0.14 s/cmd but fingerprinted in one packet; no integrity
- Single-surface LLM (ShelLM, HoneyLLM, HoneyGPT): generative, but SSH only; HoneyLLM 1.88 s/cmd; none anchors logs
- Blockchain audit logs: integrity, but not a deception sensor at all
- **This work: 5 surfaces · 0.8 ms routine / 2.33 s novel · tamper-evident**

- Caveat we state explicitly: prior figures are reproduced as published — different hardware, metrics, models. This is NOT a controlled head-to-head.
- ShelLM's 0.92 accuracy is a HUMAN-subject result (12 researchers, 226 commands) — stronger evidence than our automated judge; we do not claim parity
- We do not beat generative systems on raw novel-request latency; we are the design that avoids paying it on the ~80% of routine traffic

Limitations and future work

What we cannot yet claim, and what we will do next

Limitations (threats to validity)

- Realism is measured by PROXY — an LM judge can be miscalibrated, reward plausible-but-wrong output, share blind spots with the judged model
- Single run, one response model, one judge model
- Diversity rewards uniqueness — necessary but not sufficient (unique-but-wrong scores well); so we read diversity and realism together
- Rule-based classifier is coverage-limited by construction
- Per-service judge samples are small (n = 3 for SQL and API)

Future work

- **Human-subject detectability study — the primary next step**
  - Security practitioners, matched real vs decoy sessions, blinded, detection-rate outcome
- Redesign the AI-API decoy: tightly constrained prompt, or a non-generative fallback for structured payloads; re-evaluate
- Compare response models to quantify the realism / latency / cost frontier
- Measure classifier precision & recall on labelled traffic; test whether a learned classifier adds coverage without losing explainability
- Downstream: map intent-labelled logs to MITRE ATT&CK tactics

Conclusion — what to take away

Generative, not tabular

A language model behind five decoys yields non-enumerable, in-character replies (diversity 0.64–0.99, judged realism 4.37/5)

Tiered, so it is fast

Routine traffic at 0.8 ms — static-honeypot speed; only the novel tail pays 2.33 s

Tamper-evident evidence

Every record hashed and anchored on-chain; alteration detected without trusting the honeypot host

Honest about the weak spot

The AI-API decoy underperforms on both metrics — reported, diagnosed, and the priority for future work

Honest about the process

A benchmark bug that inflated throughput 100–1000× and a classifier that missed 75% of traffic — found, fixed, documented

For a system whose entire purpose is trustworthy evidence collection, transparent reporting is part of the contribution.

Thank you — questions?

A Multi-Protocol Cyber-Deception Framework Combining Language-Model Response Synthesis with Blockchain-Anchored Forensic Logging

Radhika G · Sisir Reddy Nellore · Pranav Sharma Chouduru · Surya Tejeswar Reddy Nagireddy · Guru Sai Prasad Reddy Bompalli · Vishnu Sathwick Rajala
Amrita School of Computing, Coimbatore · Amrita Vishwa Vidyapeetham, India
g_radhika@cb.amrita.edu · cb.sc.u4cse23129@cb.students.amrita.edu